

UnitTestDesign.jl: a proposal for the next interface

Prepared 2026-09-18 against branch feature/upgrade (package version 0.4.0). Everything marked *measured* was run against the current code with Julia 1.12.5; the probe scripts are summarized in Appendix A and the prototype in Appendix B.

1. Summary

The package does one valuable thing well: given a few representative values per argument, it picks a small set of argument combinations that still exercises every pair (or triple) of values. The simple call is fine. Everything past the simple call is where users fall off, and the first thing they fall over is constraints. This proposal makes three changes and a handful of smaller ones.

- 1. Constraints become declarations over named parameters, evaluated only on complete values.** `@forbid mode == :fast && solver != :none` replaces `disallow = (m, s, t) -> ....` The library evaluates each constraint once over the product of the parameters it mentions, turns it into a table of forbidden combinations, and never calls user code during search. Interactions that the constraints make impossible are detected, dropped from the coverage goal, and reported instead of crashing IPOG or hanging GND (both measured today).
- 2. Results become typed tuples or named tuples inside a TestCases vector that prints its own summary and is a table.** `f(case...), f(; case...), (; mode, solver) = case, DataFrame(cases),` and `@testset "$case" for case in cases` all work without glue code, and the REPL shows “10 cases, pairwise, of 81 combinations” so the tradeoff is visible.
- 3. The entry point leads with the decision, not the algorithm.** The README and docs answer “when do I reach for this instead of property-based testing, random testing, or `Iterators.product?`” in the first screen, and two new functions back the answer with numbers: `design_sizes(space)` shows what each strategy would cost, and `missing_interactions(existing_cases, space)` tells a user what their hand-written tests already miss, so they can *extend* a suite instead of replacing it.

Smaller changes: strength and stronger replace `n_way` and the index-based wayness dictionary; seeds may be partial; excursions take an explicit base case; single-valued parameters are allowed; `Counter`, `generate_tuples`, and the `Excursion` engine leave the public surface.

2. What exists today

2.1 Public surface

Exported	Role
<code>all_values, all_pairs, all_triples, all_tuples(...; n_way)</code>	covering designs at strength 1, 2, 3, or t
<code>values_excursion, pairs_excursion, triples_excursion</code>	one-, two-, three-parameter walks away from the first value of every parameter
<code>full_factorial</code>	every combination, optionally filtered
<code>IPOG, GND, Excursion</code>	engine tags passed as <code>engine =</code>

Exported	Role
<code>generate_tuples</code>	the internal dispatch point, exported by accident

Every design function shares keywords `engine`, `disallow`, `seeds`, `wayness`, `Counter` (`src/factorial_interface.jl`).

2.2 How a call flows

`all_tuples` validates the arguments and calls `generate_tuples(engine, ...)`. That function translates values into 1-based integer indices, so the engines only ever see an arity vector such as `[3, 3, 3, 3]`. Three translations happen at that boundary:

- `wrap_disallow` (`factorial_interface.jl:81`) wraps the user’s predicate so an integer vector becomes values, mapping index 0 (undecided) to nothing.
- `seeds_to_integers` looks up each seed value with `indexin`, so every seed must be complete and every value must be found.
- The result is converted back with `p[c]`, which is where index 0 turns into the `BoundsError` users see when a constraint defeats the engine.

The engines share one data structure, `MatrixCoverage` (`coverage_matrix.jl`): a matrix whose columns are the interactions still to cover, with 0 for “don’t care”, and a remain counter that partitions covered from uncovered columns. IPOG (`parameter_order.jl`) extends a design one parameter at a time; GND (`greedy_tuples.jl`) builds one case at a time from `M` random candidates; excursions and full factorial enumerate and filter. Mixed strength runs IPOG once per requested subset, feeding earlier results in as seeds (`ipog_multi_way`). Constraints are handled by asking the user’s predicate at every choice point whether a partially built case is still allowed, and by deleting directly forbidden columns from the coverage matrix (`remove_combinations!`).

2.3 What the documentation tells users

The docs are honest about the constraint problem (`docs/src/man/guide.md`, “Exclude forbidden combinations”): the predicate must tolerate nothing, and “the generator *may fail to find a solution* ... you will see the code try to access a vector at location 0.” The JuliaCon paper’s “Statement of Need” and “Comparison of Approaches” sections contain the best “when to use” writing in the project, but none of it reaches the README or the docs landing page, which open with the algorithm names.

3. Where it hurts (measured)

Probe	Today
<code>all_pairs([1,2,3], ["low","mid","high"], [1.0,3.7,4.9], [:greedy,:relax,:optim])</code>	returns <code>Vector{Vector{Any}}</code> , 10 cases
Same, <code>disallow = (a, b, c) -> a > 2 && b > 2.0</code>	<code>MethodError: no method matching isless(::Float64, ::Nothing)</code>
Count calls to a 4-parameter <code>disallow</code> during <code>all_pairs</code>	74 calls, 64 of them with at least one nothing argument

Probe	Today
Two constraints whose <i>combination</i> makes one pair impossible (Sec. 3.1), IPOG	BoundsError: attempt to access 2-element Vector{Symbol} at index [0]
Same, GND	never returns (killed after 90 s)
Same, pairs_excursion	returns 3 cases; the value :lu never appears and nothing says so
seeds = [[1, "mid", nothing]] (partial seed)	MethodError: Cannot convert an object of type Nothing to Int64
seeds = [[1, "medium", 3.7]] (typo in a seed value)	the same convert error, no mention of which value or parameter
wayness = Dict{3 => [[3:6], [25:30]]}, copied from guide.md	MethodError: Cannot convert Int64 to UnitRange{Int64}
wayness = Dict{3 => [3:6, 25:30]}	also a MethodError; only [collect(3:6), collect(25:30)] works
Pass w = Dict{3 => [[1,3,4]]} as wayness	w is mutated: afterwards Dict{2 => [[1,2,3,4]], 3 => [[1,3,4]]}
A parameter with one value	DomainError: Each argument should be a list of parameter values
n_way = 3 with two parameters	BoundsError ... at index [1:3]

3.1 The constraint problem, precisely

The running example for this document is a solver configuration:

```
mode    = [:fast, :exact]
solver  = [:none, :lu, :qr]      # :lu and :qr only make sense for :exact
tol     = [1e-3, 1e-6]          # :exact refuses the loose tolerance
```

The two rules are “fast implies no solver” and “exact implies tight tolerance”. Neither rule mentions (solver, tol), but together they make the pair (solver = :lu, tol = 1e-3) impossible: the only mode that allows :lu is :exact, and :exact forbids 1e-3. Today’s engines remove the *directly* forbidden pairs from the goal, then try to cover (:lu, 1e-3) anyway. IPOG leaves a 0 it cannot fill and crashes on the way out; GND loops forever looking for a candidate that scores. Five distinct usability defects sit on top of that correctness defect:

1. The predicate takes positional arguments. With 12 parameters, the user writes (_, _, m, _, _, s, _...) -> ... and gets it wrong silently.
2. The predicate receives nothing for undecided parameters, so every comparison other than == must be guarded. The docs explain this; users still hit it first.
3. The engine can only ask “is this partial case still allowed?”, which is a weaker question than “can this partial case still be completed?”, and the gap is exactly the implicit-infeasibility case above.
4. The polarity is inverted from how people state rules (“exact *requires* tight tolerance”), so users write the negation by hand.
5. Nothing reports what was excluded. In the excursion engine, coverage is silently lost.

The literature name for the fix is *forbidden tuples* (Yu, Lei, Kacker, Kuhn, “Constraint handling in combinatorial test generation using forbidden tuples”, ICSTW 2015; it is how NIST’s ACTS handles

constraints). The interface below is designed so that the library owns the constraint as data, which is what makes the fix possible.

4. Design principles

- **One concept per level.** A one-line call stays one line. Naming parameters is one added concept and unlocks everything else. Constraints, seeds, strength, and inspection each add exactly one more.
- **The library owns constraints as data, not as opaque callables.** It can then evaluate them safely, detect infeasibility, print them in error messages, serialize them, and hand them to a solver later.
- **User code is never called with partial information.** No `nothing`, `no missing`, `no index 0`.
- **Nothing is dropped silently.** Infeasible interactions, rejected seeds, and unused values are reported.
- **The result is a first-class Julia value.** Typed, iterable, a table, and self-describing at the REPL.
- **Guarantees are stated in the docstrings:** every returned case satisfies every constraint; every feasible t-way interaction is covered; seeds appear first, in order; IPOG is deterministic.

5. The proposed interface, by level

Level 0: one line, positional (unchanged shape, better result)

```
using UnitTestDesign
```

```
cases = all_pairs([1, 2, 3], ["low", "mid", "high"], [1.0, 3.7, 4.9])
for (n, level, tol) in cases
    @test integrate(n, level, tol) reference(n, level, tol)
end
```

`cases` is a `TestCases{Tuple{Int64, String, Float64}}`, an `AbstractVector`. `case[1]` still works, so existing loops keep working, but the element type is concrete and `f(case...)` is type-stable. Single-valued parameters are allowed (a fixed argument is just an argument with one representative). The same positional form exists for `all_values`, `all_triples`, `covering`, `full_factorial`, and `excursions`.

Level 1: named parameters

Naming is the one concept that unlocks constraints, partial seeds, subset strength, tables, and readable test names. Parameters are `name => values` pairs, in the order the function under test takes them:

```
cases = all_pairs(:n => [1, 2, 3], :level => ["low", "mid", "high"], :tol => [1.0, 3.7, 4.9])

@testset "integrate $(case)" for case in cases
    (; n, level, tol) = case                # property destructuring
    @test integrate(n, level, tol) reference(; case...) # or keyword splat
end
```

cases is now a `TestCases{@NamedTuple{n::Int64, level::String, tol::Float64}}`. A `NamedTuple` literal (`n = [1, 2, 3]`, `level = [...]`) is accepted as a single positional argument as well, since it is the natural Julia spelling.

When the same parameters feed several designs, or need to be inspected, they live in a `ParameterSpace`:

```
space = ParameterSpace(
    :mode    => [:fast, :exact],
    :solver  => [:none, :lu, :qr],
    :tol     => [1e-3, 1e-6];
    constraints = [
        @forbid(mode == :fast && solver != :none),
        @require(mode == :fast || tol < 1e-4),
    ],
)
quick    = all_pairs(space)           # CI
nightly  = all_triples(space)         # scheduled
```

Every design function accepts either a `ParameterSpace` or the pairs directly, with `constraints =` available in both places.

Level 2: constraints

Three spellings, from most to least common:

1. A literal forbidden combination. No logic, no macro.

```
forbid(mode = :fast, solver = :lu)
```

2. An expression over parameter names. Bare identifiers are parameter names;

`\$x` pulls a variable in from the surrounding scope.

```
@forbid mode == :fast && solver != :none
@require mode == :fast || tol < $threshold
@forbid solver in (:lu, :qr) && !isfinite(tol)
```

3. A function, when the logic is too big for an expression. Names are explicit.

```
forbid(:mode, :solver) do mode, solver
    mode == :fast && solver != :none
end
```

`@require e` is exactly `@forbid !(e)`; having both verbs keeps the polarity visible in each rule. All three construct the same `Constraint` value, which remembers its scope (the parameter names it mentions), its predicate, and its source text for printing.

Semantics. For each constraint, the library evaluates the predicate once for every combination of values of the parameters in its scope (the product of those domains, which is 6 evaluations for `(mode, solver)` above) and records the combinations that are forbidden. From then on:

- A complete case is valid iff none of its projections onto a scope is a forbidden combination.
- A partial case is *rejected* if it already contains a forbidden combination and *dead* if no valid completion exists; deadness is decided by a backtracking completion search over the remain-

ing parameters, not by asking the user.

- The coverage goal is every strength-t interaction that occurs in at least one valid complete case. Directly forbidden interactions are excluded trivially; implicitly infeasible ones are found by the completion search and listed in the result's report.

The user's predicate is never called during search and never with anything but values drawn from the declared domains, so `tol < 1e-4` needs no guard. The cost is the product of the scope's domains; a constraint mentioning ten four-valued parameters is about a million evaluations, so the library warns above a threshold and suggests splitting the rule. Real constraints mention two to four parameters.

Errors at construction, not deep in the engine:

`ArgumentError: @forbid(mode == :fast && solvr != :none) mentions `solvr`, which is not a parameter. Parameters are (:mode, :solver, :tol). If `solvr` is a variable, write `$solvr`.`

Dependent parameters are the most common real constraint ("solver only applies when `mode == :exact`"). The recipe is a sentinel value plus one `@require`, which the design handles without special support:

```
:solver => [:none, :lu, :qr]
@require mode == :exact || solver == :none
```

Reporting. Running the running example gives (prototype output, Appendix B):

```
5 test cases · pairwise · 3 parameters · full factorial would be 12 (5 valid)
mode      solver  tol
:fast     :none   0.001
:exact    :none   1.0e-6
:exact    :lu     1.0e-6
:exact    :qr     1.0e-6
:fast     :none   1.0e-6
```

Constraints forbid 3 pairs directly and make 2 more unreachable: (solver = :lu, tol = 0.001),

The last line is the one that today is a `BoundsError`. It is also useful in its own right: a user who did not intend (solver = :lu, tol = 1e-3) to be untestable finds out here.

Level 3: seeds, strength, engines, and excursions

Seeds are cases that must appear. They are named tuples, may be partial, and appear first in the output in the order given:

```
all_pairs(space; seeds = [
  (mode = :exact, solver = :lu, tol = 1e-6),    # the production configuration
  (mode = :fast, ),                             # partial: the engine fills the rest
])
```

A seed that violates a constraint or names a value outside its domain is an error that names the seed, the parameter, and the offending value. An existing `TestCases` value is accepted as seeds, which is how a suite is extended (Sec. 6.3).

Strength replaces `n_way`, and **stronger** replaces `wayness`:

```
covering(space; strength = 2) # == all_pairs(space)
covering(space; strength = 2, stronger = [(:mode, :solver, :tol) => 3])
```

Subsets are named, not indexed, so the guide example `Dict{3 => [[3:6], [25:30]]}` becomes `stronger = [(:c, :d, :e, :f) => 3, (:y, :z) => 3]` and cannot be mistyped into a convert error. `strength` greater than the number of parameters is an `ArgumentError`; `strength` equal to it is a full factorial. `all_values`, `all_pairs`, and `all_triples` remain as the names people search for. `all_tuples` remains as a deprecated alias of `covering`.

Engines keep their literature names but hide their knobs behind readable ones:

```
covering(space; engine = IPOG()) # default, deterministic
covering(space; engine = GND(rng = Xoshiro(1), candidates = 50)) # `candidates` was `M`
```

Excursions get an explicit base case. Today the base is silently “the first value of every parameter”, which is the single most surprising fact about `values_excursion`:

```
excursions(space; from = (mode = :exact, solver = :lu, tol = 1e-6), distance = 1)
excursions(space; distance = 2) # base defaults to first values, and the docstring says so
```

`distance` is how many parameters may differ from the base at once, so `values_excursion`, `pairs_excursion`, and `triples_excursion` become aliases for distances 1, 2, and 3 and stay exported. Because excursions are filtered by the same constraint machinery, the report says which values never appear.

Full factorial takes the same space and constraints and reports the count before and after constraints.

Level 4: inspection

```
design_sizes(:n => [1, 2, 3], :level => ["low", "mid", "high"], :tol => [1.0, 3.7, 4.9], :kind
# strategy          cases          (measured with today's engines)
# full_factorial    81              (with constraints: total and valid counts)
# all_values        3
# all_pairs         10
# all_triples       31
# excursions(1)     9
# excursions(2)     33
```

```
coverage(cases, space; strength = 2) # (covered = 11, feasible = 11, missing = [])
missing_interactions(handwritten, space) # [(solver = :qr, tol = 1.0e-6), ...]
report(cases) # the summary the REPL prints, plus the unreachable
```

`design_sizes` is the “should I bother?” function; it answers the user’s question before they commit. `missing_interactions` is the on-ramp for a project with an existing suite: convert the existing calls to named tuples, ask what pairs they miss, and pass them as seeds to get the minimal extension.

Reference: proposed signatures

```
ParameterSpace(pairs::Pair{Symbol}...; constraints = Constraint[])
ParameterSpace(nt::NamedTuple; constraints = Constraint[])
```

```

covering(space; strength = 2, stronger = Pair[], constraints = [], seeds = [], engine = IPOG())
all_values(space; kw...)           # strength = 1
all_pairs(space; kw...)            # strength = 2
all_triples(space; kw...)          # strength = 3
excursions(space; from = nothing, distance = 1, constraints = [], seeds = [])
full_factorial(space; constraints = [])
# every function above also accepts `pairs::Pair{Symbol}...` or `values::AbstractVector...` in

forbid(; name = value, ...)        -> Constraint
forbid(names::Symbol...) do values... end -> Constraint
require(names::Symbol...) do values... end -> Constraint
@forbid expr; @require expr        -> Constraint

```

```

design_sizes(space; strengths = 1:3, distances = 1:2)
coverage(cases, space; strength = 2)
missing_interactions(cases, space; strength = 2)
report(cases)

```

```

struct TestCases{T} <: AbstractVector{T}  # T is a Tuple or NamedTuple type
IPOG(); GND(; rng = Random.default_rng(), candidates = 50)

```

Removed from the public surface: `generate_tuples`, `Excursion`, `Counter`, `n_way`, `wayness`, `disallow` (kept for one minor version as a deprecated keyword that wraps the function form with nothing-safe semantics, see Sec. 9).

6. The result type

6.1 TestCases

```

struct TestCases{T} <: AbstractVector{T}
    cases::Vector{T}
    space::ParameterSpace
    strategy::Symbol           # :covering, :excursion, :full_factorial
    strength::Int              # or distance, for excursions
    seeded::Int                # how many leading cases came from seeds
    infeasible::Vector{NamedTuple} # interactions dropped from the goal
end

```

It is a vector, so everything that works on a vector works on it. Because it is an `AbstractVector` of `NamedTuples` it is already a `Tables.jl` row table, so `DataFrame(cases)` and `CSV.write("cases.csv", cases)` work with no dependency added here. `show` prints the one-line summary and an aligned table, truncated like a `DataFrame`. The metadata is what `report`, `coverage`, and the deprecation shims read.

6.2 REPL output as documentation

```

julia> all_pairs(:n => [1, 2, 3], :level => ["low", "mid", "high"], :tol => [1.0, 3.7, 4.9], :
10 test cases · pairwise · 4 parameters · full factorial would be 81
  n  level  tol  kind

```

```

1  "low"    1.0  :greedy
2  "mid"    3.7  :relax

```

“10 of 81” is the whole pitch of the package, and it should be the first thing a user sees.

6.3 Extending an existing suite

```

existing = [(mode = :fast, solver = :none, tol = 1e-3), (mode = :exact, solver = :lu, tol = 1e-3)]
missing_interactions(existing, space)           # what the hand-written tests miss
all_pairs(space; seeds = existing)              # the smallest set that keeps them and fixes them

```

7. Positioning: telling people when to use it

7.1 The decision in one table

This belongs at the top of the README and the docs landing page, before any function name.

Your situation	Reach for
You can list a few representative values per argument, the function has several arguments, and you suspect bugs live in <i>combinations</i> of arguments (an if on one option inside a branch on another).	A covering design: <code>all_pairs</code> , <code>all_triples</code> .
Same, and each run is cheap and the product is small.	<code>full_factorial</code> or <code>Iterators.product</code> .
You have one known-good configuration and want to know which single (or paired) changes break it.	<code>excursions</code> .
You can <i>generate</i> values but not enumerate representatives (strings, trees, arbitrary floats), runs are cheap, and you want failing inputs shrunk.	Property-based testing (<code>Supposition.jl</code> , <code>PropCheck.jl</code>).
You already have hand-written cases and want to know what interactions they miss.	<code>missing_interactions</code> , then <code>seeds</code> .
Both apply.	Use covering designs to pick <i>which equivalence class</i> each argument draws from, and a generator to draw within it.

7.2 The one-sentence explanation

Anchor to something users already know: `Iterators.product(a, b, c)` is every combination; `all_pairs(a, b, c)` is the smallest subset of it in which every pair of values still occurs together. Random sampling gets there too, but slowly and without a guarantee (measured, uniform random cases, 200 trials):

parameters × values	designed pairwise cases	random cases needed to cover every pair, median [10%, 90%]	pairs covered by that many random cases
5 × 4	16	84 [65, 116]	64%
10 × 4	28	106 [87, 131]	84%
10 × 8	113	530 [461, 633]	83%
40 × 4	45	151 [133, 183]	95%
40 × 8	166	709 [632, 845]	93%

The honest framing: random testing spends four to six times as many runs to reach the same pairwise guarantee, and property-based testing spends them to buy something different (generation and shrinking). Designed cases win when runs are expensive or the argument space is large, and lose when the user cannot name representatives. The paper already says this; the README should.

7.3 Docs restructure

1. **Home:** the table above, the one-sentence explanation, the 10-of-81 example with its REPL output, and the PBT bridge in three lines.
2. **Tutorial** (replaces Guide): levels 0 through 4 in order, each a runnable @example block, ending with “extend an existing suite”.
3. **Constraints:** the three spellings, the semantics paragraph, the dependent-parameter recipe, and what the report means.
4. **Choosing strength and strategy:** the `design_sizes` table for a real example, when to raise strength, when excursions beat coverings.
5. **Engines and algorithms:** the existing IPOG and greedy pages, with the forbidden-tuple and completion-search additions.
6. **Reference.**

The paper’s “How to Use” section (equivalence classes, oracles, invariants) belongs in the tutorial, not only in the PDF.

8. What the interface assumes of the engines

The interface is only honest if the engines deliver these. Each is stated as a contract, with what it costs to meet it. The prototype in Appendix B meets the first three on top of the *current* IPOG code, which is the evidence that the list is realistic rather than aspirational.

- **E1. Constraints arrive as forbidden-combination tables, never as callables.** A preprocessing step tabulates each Constraint over its scope. The engines’ existing integer-vector predicate is kept internally but is now generated by the library and is partial-safe by construction (a table only fires once its whole scope is assigned). No engine change.
- **E2. Implicitly infeasible interactions are detected and dropped from the goal.** Before generation, each target interaction is checked for a valid completion by backtracking with forward checking over the remaining parameters. Cost is negligible with few constraints: the 12-parameter, four-constraint example in Appendix B takes 0.7 s end to end, most of it in IPOG. Later optimization: derive minimal forbidden tuples once (Yu et al.

2015) so per-interaction search is rarely needed. Prototype: done on top of `ipog_multi` by adding the dead interactions to the predicate.

- **E3. Engines never leave an unfillable slot.** Replace the greedy `fill_remaining_missing_values_fill` and `choose_last_parameter_filter!` fallbacks with backtracking completion. With E2 in place, this only triggers for higher-order implicit infeasibility (a partial case whose every pair is feasible but whose triple is not), which is rare and cheap. This is what makes “index 0” impossible by construction rather than by luck.
- **E4. GND cannot spin.** Its candidate loop (`while trial_cnt < M`) must cap attempts and fall back to backtracking completion; with E2 removing unreachable goals it also terminates by the same argument as today.
- **E5. Partial seeds and single-valued parameters.** Measured: `ipog`, `ipog_multi`, and the greedy generator all accept an arity of 1 and reach full coverage, and `ipog_multi` given the partial seed `[1, 0, 2]` fills the zero and keeps the seed first. Only the translation layer (`seeds_to_integers`, the `< 2` checks in `all_tuples`) forbids them. Small change.
- **E6. Coverage is reportable.** `test_coverage` and `tuples_in_trials` already exist in `coverage_set.jl`; they need to be exposed through `coverage` and `missing_interactions` in value space rather than index space.
- **E7. Determinism and reproducibility as stated.** IPOG is deterministic today. Constraint tabulation and dead-interaction detection must iterate in a fixed order so the report is stable across runs.
- **E8. Room to grow, without changing the interface.** Because a `Constraint` keeps its expression, a package extension can translate the comparison-and-boolean subset to Z3 (the sketch in `z3_example.jl`) or a SAT solver for large scopes, and the same strings can be read from TOML for the command-line branch (`feature/command-line`), which today has no way to express `disallow` at all. Neither is needed for the first release.

Suggested order: E1, E2, E5, E6 (the translation layer, no engine surgery), then E3 and E4 (engine surgery), then E7 checks, then E8 as separate projects.

9. Migration

Today	Proposed	Notes
<code>all_pairs(v1, v2, v3)</code> returning <code>Vector{Vector{Any}}</code>	same call, returns <code>TestCases{Tuple{...}}</code>	breaking only for code that mutates result rows or depends on <code>Vector{Vector}</code>
<code>all_tuples(...; n_way = k)</code>	<code>covering(...; strength = k)</code>	<code>all_tuples</code> and <code>n_way</code> deprecated one minor version
<code>disallow = (a, b, c) -> ...</code>	<code>constraints = [@forbid ...]</code>	<code>disallow</code> kept one version: wrapped as a full-scope function-form constraint, so it is evaluated only on complete cases and the nothing contract disappears

Today	Proposed	Notes
<code>wayness = Dict{3 => [[3, 4, 5, 6]]}</code>	<code>stronger = [(:c, :d, :e, :f) => 3]</code>	positional call: <code>stronger = [(3, 4, 5, 6) => 3]</code> accepted
<code>seeds = [[1, "mid", 3.7, :relax]]</code>	<code>seeds = [(n = 1, level = "mid", tol = 3.7, kind = :relax)]</code>	positional call still accepts vectors/tuples; partial seeds new
<code>values_excursion(...)</code>	<code>excursions(...; distance = 1)</code>	old names stay as aliases; from = new
<code>engine = GND(M = 50)</code>	<code>engine = GND(candidates = 50)</code>	M deprecated
<code>Counter = Int8</code>	removed	choose internally from arity
<code>generate_tuples, Excursion</code>	unexported	

Version: this is a 1.0 candidate. The return-type change is the only break that cannot be shimmed, and it is the change most worth making.

10. Open questions for the author

1. **Names.** `ParameterSpace`, `covering`, `stronger`, `forbid/require`, `excursions(; from, distance)`. Alternatives considered: `Factors/levels` (DoE vocabulary, foreign to unit testers), `design` (collides with user variables), `constraints = [...]` versus `where = ...` (kept `constraints`, the term every other CIT tool uses).
2. **Should the positional form support constraints at all?** This proposal says no: constraints need names. A positional user adds names when they add a constraint, which is one honest step up.
3. **Labels for values.** `:x => ["tiny" => 1e-9, "huge" => 1e9]` would make test names and reports readable for float and struct values. It adds a concept; deferred.
4. **Keep GND?** It gives the same guarantees, is slower, and is less predictable. It sometimes finds smaller designs, which matters when each run costs hours. Keep, but do not mention it before the “Engines” page.
5. **Generators as values.** The PBT bridge could be a helper that draws from generator values at iteration time. Values that are functions are also legitimate arguments to `test([sin, cos])`, so nothing automatic; document the pattern instead.
6. **Command line.** With constraints expressible as strings of the safe subset, the TOML format from `feature/command-line` becomes complete. Worth reviving after 1.0.

Appendix A. Probe results

Script: `probe.jl` (interface probes), `probe_gnd.jl` (GND hang under a 90 s alarm), `random_vs_design.jl` (Sec. 7.2 table), run with `julia --project=. <script>` against the current working tree. Selected raw output:

```
== output type: OK (Vector{Vector{Any}}, 10)
== disallow with > on nothing: ERR MethodError :: no method matching isless(::Float64, ::Nothing)
== count disallow calls with nothing: OK (calls = 74, with_nothing = 64)
== IPOG implicit infeasible: ERR BoundsError :: attempt to access 2-element Vector{Symbol} at ...
```

```

== pairs_excursion implicit infeasible: OK (3, [[:fast, :none, 0.001], [[:fast, :none, 1.0e-6],
== partial seed with nothing: ERR MethodError :: Cannot `convert` an object of type Nothing to
== seed value not in list: ERR MethodError :: Cannot `convert` an object of type Nothing to an
== wayness Dict(3 => [[3:6], [25:30]]) as in guide: ERR MethodError :: Cannot `convert` an obj
== wayness Dict(3 => [3:6, 25:30]): ERR MethodError :: Cannot `convert` an object of type Vect
== wayness Dict(3 => [collect(3:6), collect(25:30)]): OK 76
== wayness dict mutated?: OK Dict(2 => [[1, 2, 3, 4]], 3 => [[1, 3, 4]])
== single-valued parameter: ERR DomainError with ["only"]: Each argument should be a list of p
== n_way > nparams: ERR BoundsError :: attempt to access 2-element Vector{Int64} at index [1:3
GND with implicitly infeasible tuple: killed by alarm after 90 s (exit status 142)

```

Appendix B. Prototype of the constraint layer on today's engine

About 80 lines, run against the current `ipog_multi` without modifying it. The pieces map onto Sec. 5 and Sec. 8 directly.

*# Scope capture: every bare identifier that is not in call position is a parameter name;
`\$x` is left alone and resolves in the caller's scope.*

```

function free_symbols!(syms, ex)
    if ex isa Symbol
        push!(syms, ex)
    elseif ex isa Expr
        if ex.head == :call
            foreach(a -> free_symbols!(syms, a), ex.args[2:end])
        elseif ex.head == :$
            nothing
        elseif ex.head == :.
            free_symbols!(syms, ex.args[1])
        else
            foreach(a -> free_symbols!(syms, a), ex.args)
        end
    end
    syms
end

macro forbid(ex)
    syms = unique(free_symbols!(Symbol[], ex))
    fn = Expr(:->, Expr(:tuple, syms...), uninterpolate(ex))
    :(Forbid($(Tuple(syms)), $(esc(fn)), $(string(ex))))
end

# Tabulate over the scope's domains: the only place the user's predicate runs.
function tabulate(c::Forbid, names, domains)
    scope = [findfirst(==(n), names) for n in c.names]
    doms = [domains[i] for i in scope]
    table = Set(combo for combo in Iterators.product((1:length(d) for d in doms)...))
        if c.f(ntuple(k -> doms[k][combo[k]], length(doms)))...))
    (scope = scope, table = table)
end

```

```

violates(v, t) = all(v[i] != 0 for i in t.scope) && ntuple(k -> v[t.scope[k]], length(t.scope))

# Backtracking completion: can this partial assignment (0 = undecided) become a valid case?
function completable(v, arity, tabs)
  any(t -> violates(v, t), tabs) && return false
  i = findfirst(==(0), v); i === nothing && return true
  w = copy(v)
  for val in 1:arity[i]
    w[i] = val
    completable(w, arity, tabs) && return true
  end
  false
end
infeasible(arity, t, tabs) = (allc = all_combinations(arity, t);
  [allc[:, j] for j in axes(allc, 2) if !completable(allc[:, j], arity, tabs)])

# Drive the existing engine with a partial-safe predicate that also rejects dead partial cases
dead = infeasible(arity, 2, tabs)
pred = v -> any(t -> violates(v, t), tabs) || any(d -> all(d[i] == 0 || d[i] == v[i] for i in
result = UnitTestDesign.ipog_multi(arity, 2, pred, zeros{Int, length(arity)}, 0))

```

Output for the running example (the one that throws `BoundsError` today):

```

forbidden tables:
  scope [:mode, :solver] forbids [(1, 2), (1, 3)] (6 evaluations)
  scope [:mode, :tol] forbids [(2, 1)] (4 evaluations)
total 2-way interactions: 16; uncoverable: 5 (3 directly forbidden, 2 only by implication)
  implied: (solver = :lu, tol = 0.001); (solver = :qr, tol = 0.001)
full factorial: 12 (5 valid)
design (5 cases):
  (mode = :fast, solver = :none, tol = 0.001)
  (mode = :exact, solver = :none, tol = 1.0e-6)
  (mode = :exact, solver = :lu, tol = 1.0e-6)
  (mode = :exact, solver = :qr, tol = 1.0e-6)
  (mode = :fast, solver = :none, tol = 1.0e-6)
all cases satisfy constraints: true
pairs uncovered: 5 of 16 (exactly the uncoverable ones)

12-parameter example (arities 3..5, four constraints, one of them a triple):
total 2-way interactions: 590; uncoverable: 4 (3 directly forbidden, 1 only by implication: (p
full factorial: 331776 (207360 valid); design: 20 cases in 0.7 s

```

The `$threshold` interpolation was exercised by the second constraint (`@require(mode == :fast || tol < $threshold)`), which the macro reports as `@forbid(!(mode == :fast || tol < $threshold)) # over (:mode, :tol)`.